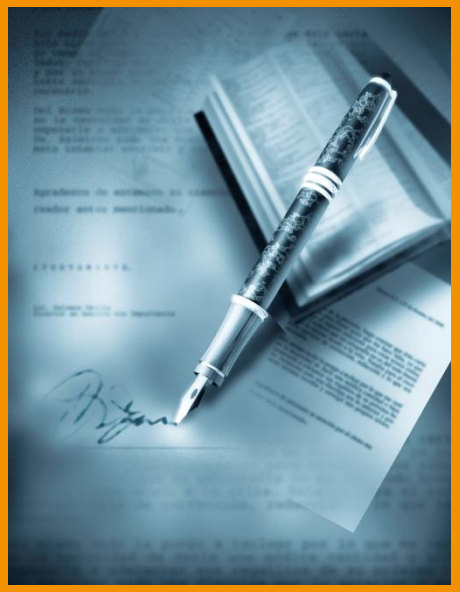
IT-Security

Technische Hochschule Rosenheim

Chapter 3:
Checksums and Digital Signatures



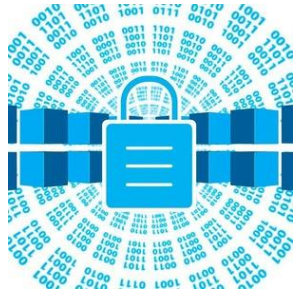
1. Hash functions
2. Message Authentication Code MAC
3. Digital Signatures
4. Practical aspects of digital signatures
5. PKI

Prof. Dr. Reiner Hüttl, TH Rosenheim, © 2026, 15.03.2026

What do we want to learn?

- What is the meaning of hash functions?
- What is a cryptographic hash function, such as MAC?
- How does a digital signature work?
- What is the legal significance of electronic signatures?
- What do you have to consider in the practical implementation?
- What are the components of a PKI?
- How is a certificate structured, how do you verify it?

How do I ensure the integrity of my data?



- Problem:
  - I have a lot of data and want a short checksum to make sure the data has not been modified.
- Possible solutions
  - Hashing
  - Cryptographic hashing
  - Digital signatures

Not safe against tampering!

Technische Hochschule Rosenheim

Chapter 3.1: Hash Functions

- Secure Hash Functions
- SHA-3
- Password Hashing

Hash Functions

BUT: Hash functions are not encryption!

- One-way encryption functions:
  Mapping of variable length input to "unique" output with a fixed length.
- Serves as checksum, fingerprint, message digest
- Properties
  - small change in message results in large change in hash value
  - message cannot be reconstructed from hash value (function cannot be inverted)
  - hash value as unique as possible
    collision resistant: it is practically impossible to find two values x1 and x2 with f(x1) = f(x2)
  - hash value can be calculated fast and by everyone

Not recomended anymore

- Algorithms: MD5 (128 bit), SHA-1 (160 bit),
  SHA-2 (SHA-256 (256 bit), SHA-384, SHA-512, SHA-224)
  SHA-3

Security of hash functions

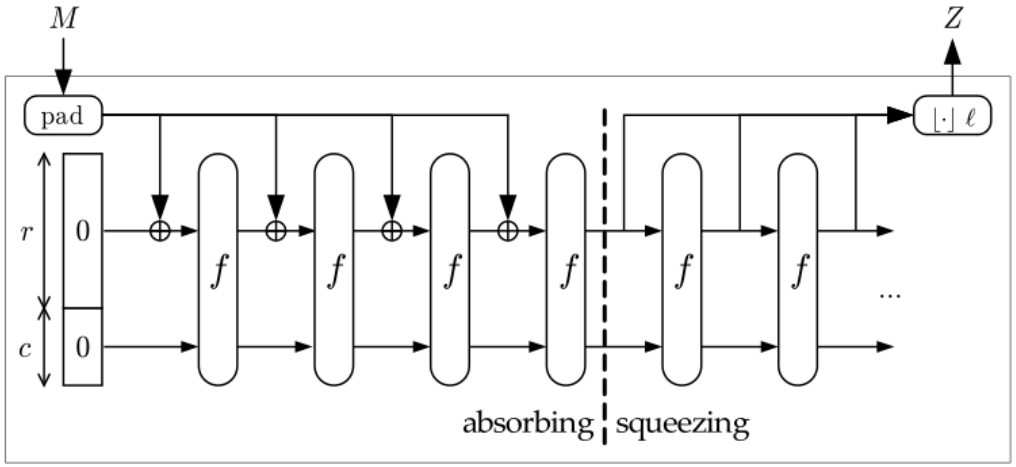
- Insecure "classic" methods: MD5, SHA1, RIPEMD
- Recommendations and key lengths for cryptographic methods by BSI

Methods	Recommended algorithms
cryptographic strong hash functions	SHA-256, SHA-512/256, SHA-384 und SHA-512, SHA3-256, SHA3-384, SHA3-512
MAC methods	HMAC>= 128, CMAC>=128, GMAC>=128, KMAC>=256
Signature methods	RSA 3000, DSA 3000 , ECDSA 250, Merkle signatures

https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=6

New hash standard since 2012: SHA-3 (Keccak)

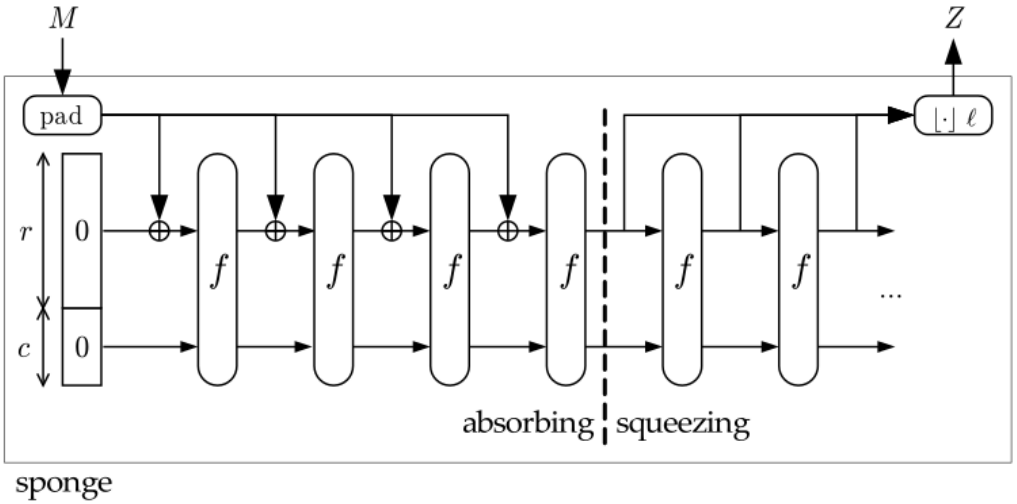
- Winner from a competition by the NIST (National Institute of Standards and Technology)
  http://csrc.nist.gov/groups/ST/hash/sha-3/
  https://keccak.team/index.html
- Important requirement: hash must be quickly computable
- Output size is variable (224, 256, 384, 512 bit)
- Security can be influenced by capacity using a sponge function
  (more security -> less performance)



sponge

Quelle: http://sponge.noekeon.org/

Details of SHA-3 (Keccak)



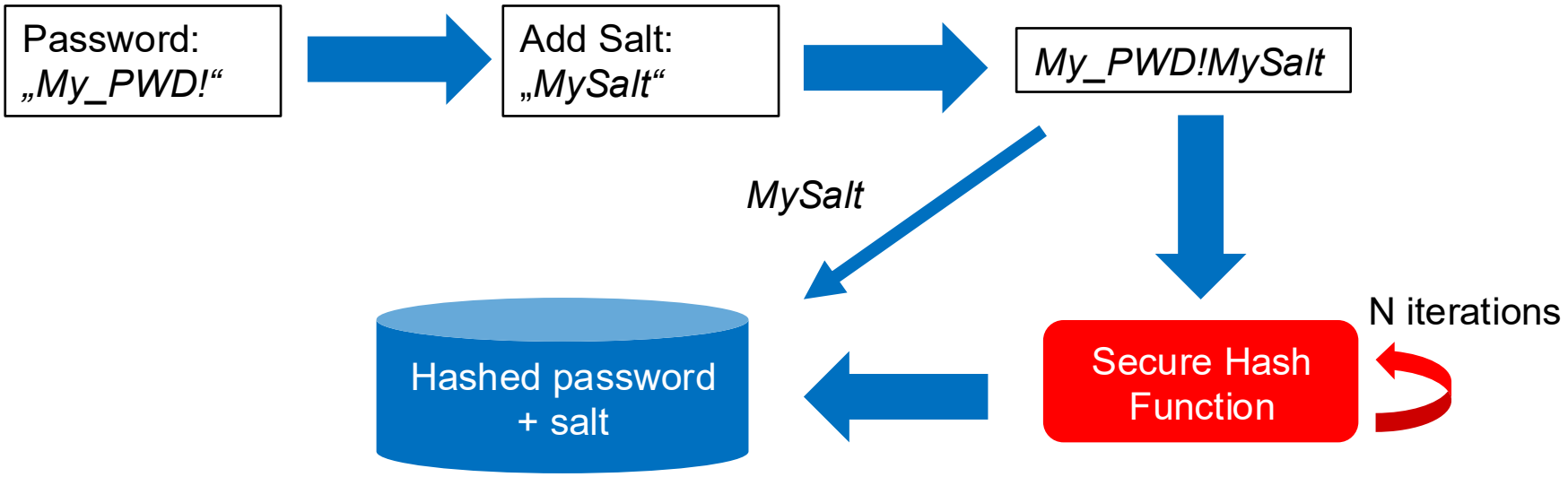
Quelle: http://sponge.noekeon.org/

- Absorbing:
  - state is divided into two parts: Capacity (c bits) and Bit-Rate (r bits).
  - input blocks M are padded (filled up)
  - r-bit part is XORed with state, c-bit part remains unchanged
  - then state is mixed with f (Keccak permutation).
- Squeezing
  - Hash value is extracted from r-bit portion of state
  - If length is not sufficient, state is permuted several times and further outputs are extracted after each permutation.
- Visualization see CrypTool 2 (https://www.cryptool.org/de/ct2/)

Hash functions for password hashing

- Problems
  - normal Hash functions are very fast and therefore support brute force attacks
  - same password results in same hash value
- Criteria for Secure Hash functions https://www.password-hashing.net/cfh.html
  - add cost parameter to tune time and/or space usage
  - provide cryptographic security (no speed up for hackers possible)
  - combine it with a **Salt** to produce different hash values for same password
    Salt: random data as additional input for the hash function,
    generate new salt for each password
- Secure Password Hashing Algorithms
  - Bcrypt, Scrypt, Argon2, PBKDF2

Process for secure password hashing



Technische Hochschule Rosenheim

Chapter 3.2: Message Authentication Code

- Data Authentication
- MAC
- HMAC, CBC-MAC, Poly1305 MAC

Data authentication

- Data authentication means cryptographic procedures that guarantee that transmitted or stored data has not been modified by unauthorized persons.
  - it is based on cryptographic keys that are used to calculate checksums.
  - there are symmetric and asymmetric procedures
- Two security goals can be achieved with data authentication
  - ensuring the **integrity** of data
  - securing the **non-repudiation** of a message
    → this is only possible with digital signatures

https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=6

Message Authentication Code (MAC)

- A hash function that additionally uses a secret key
- The key is known only to the two communication partners
- This enables the detection of unauthorized modifications (**integrity**)
- This enables the verification if data have an authentic origin (**authenticity**)

First attempt for a MAC

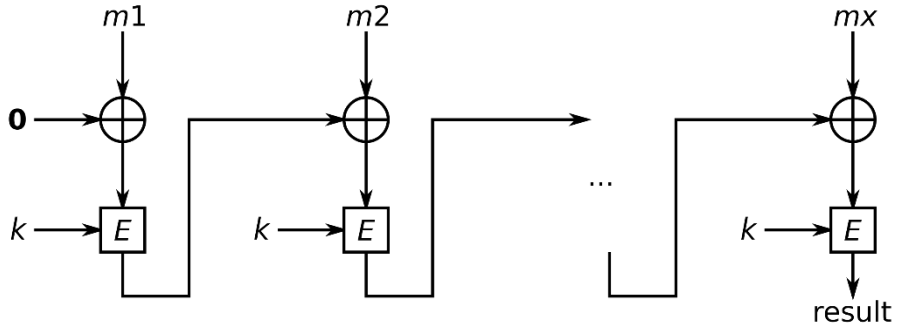
- Simple concatenation *hash(Secret || Daten)* = MAC
- Problem:
  - if the attacker knows *length(secret || data)*
  - then he can compute for some hash functions *hash(secret || data || more data)* **without** knowing the secret!
- This is called: **Length extension attack**
  https://en.wikipedia.org/wiki/Length_extension_attack
- Vulnerable are e.g. SHA1, SHA2, not vulnerable is SHA3
- Problem: you must create a hash, which can only be calculated with a secret.
  - one possible solution: HMAC

HMAC (Hashed MAC)

- HMAC is a popular MAC
- $HMAC\,(K,m) = H\,((K \oplus opad) \parallel H\,((K \oplus ipad) \parallel m))$
  - K is the secret key
  - block size is 64 bytes: either pad K with zeros or reduce to 64 bytes with a hash function.
  - opad, ipad are constants in block size
- A video illustrating HMAC
  https://www.youtube.com/watch?v=BjInMA-b8ZE

CBC-MAC (Cipher Block Chaining MAC)

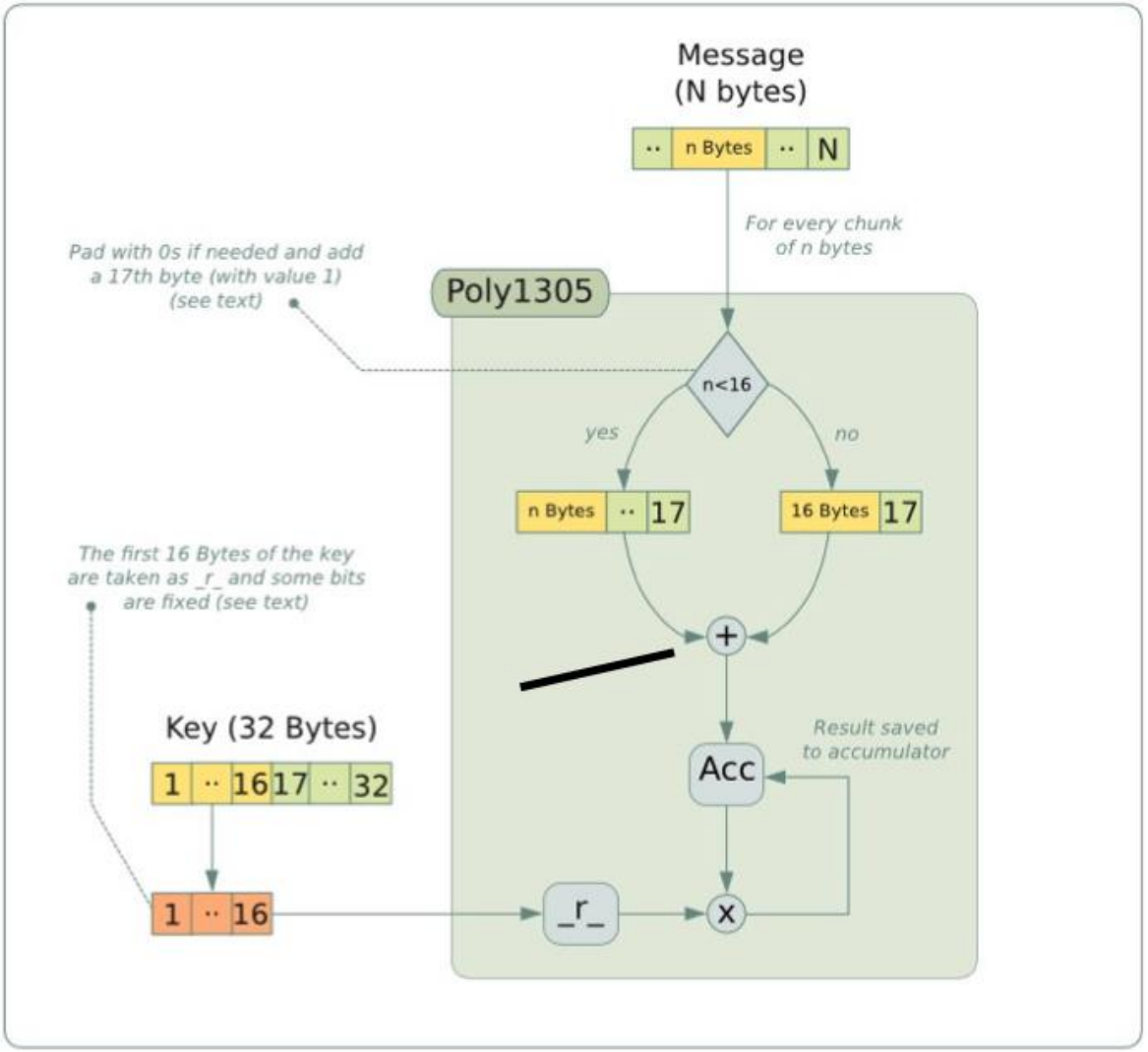
- CBC-MAC is a technique to calculate a MAC using a block cipher method.
- It uses block cipher method in CBC mode
- The IV is set to 0, the MAC is the last encrypted block
- $c_1 = (m_1 \oplus 0) \oplus K$ $\quad$ $c_x = (m_x \oplus c_{x-1}) \oplus K$
- CBC-MAC is only safe for messages of fixed/known length, otherwise a length extension attack is possible.



Quelle: By Benjamin D. Esham (bdesham) - Own work based on: Cbcmac.png by en:User:PeterPearson.Own work by bdesham using: Inkscape., Public Domain, https://commons.wikimedia.org/w/index.php?curid=2277179

Poly1305 MAC

- Poly1305 is a MAC which uses a 32 Byte secret key and generates a 16 Byte authenticator
- Poly1305 is faster than HMAC



Quelle: https://www.adalabs.com/adalabs-chacha20poly1305.pdf

Applications and limitations of MAC

- Modern ciphering methods combine encryption with MAC calculation
  - Counter Mode **CTR with CBC-MAC**
  - **GCM** (see chapter 2: GCM is a cipher with MAC calculation)
  - **ChaCha20 with Poly1305**
- Application of MAC
  - attack detection for file systems (Filesystem Intrusion Detection)
  - securing software packages (patch, update)
  - securing of communication protocols (e.g. TLS)
- A MAC secures the integrity and authenticity of a message
  - but it lacks the evidential value for non-repudiation
  - it cannot be verified by a third-party authority
  - it is based on symmetric cryptography
  - it lacks certificates

Technische Hochschule Rosenheim

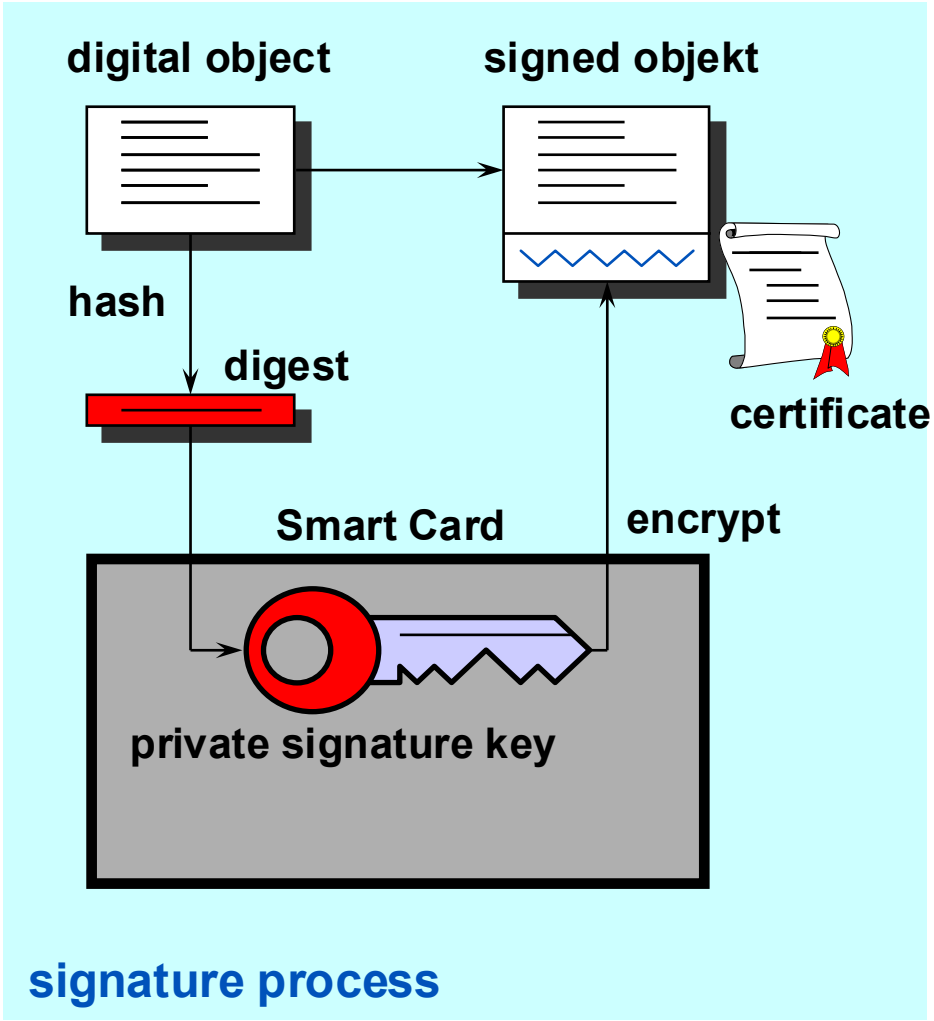
Chapter 3.3: Digital Signatures

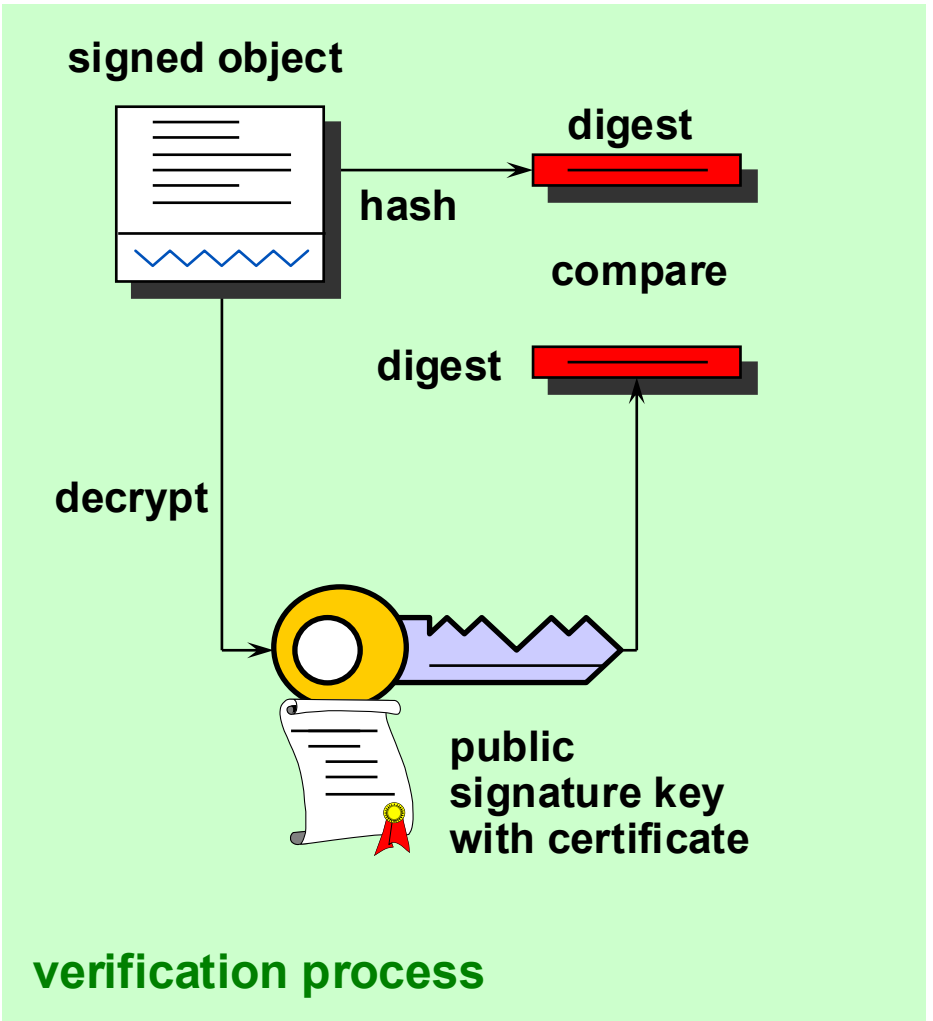
- Signature procedure
- XML Signatures

Digital signatures ensure the integrity of data and prove authenticity

- A Digital Signature contains
  - time from a timestamp service
  - person or place (server name)
  - signed digest of the document
- A Digital Signature proves
  - integrity of the document (what)
  - who signed and when

The procedure of the Digital Signature

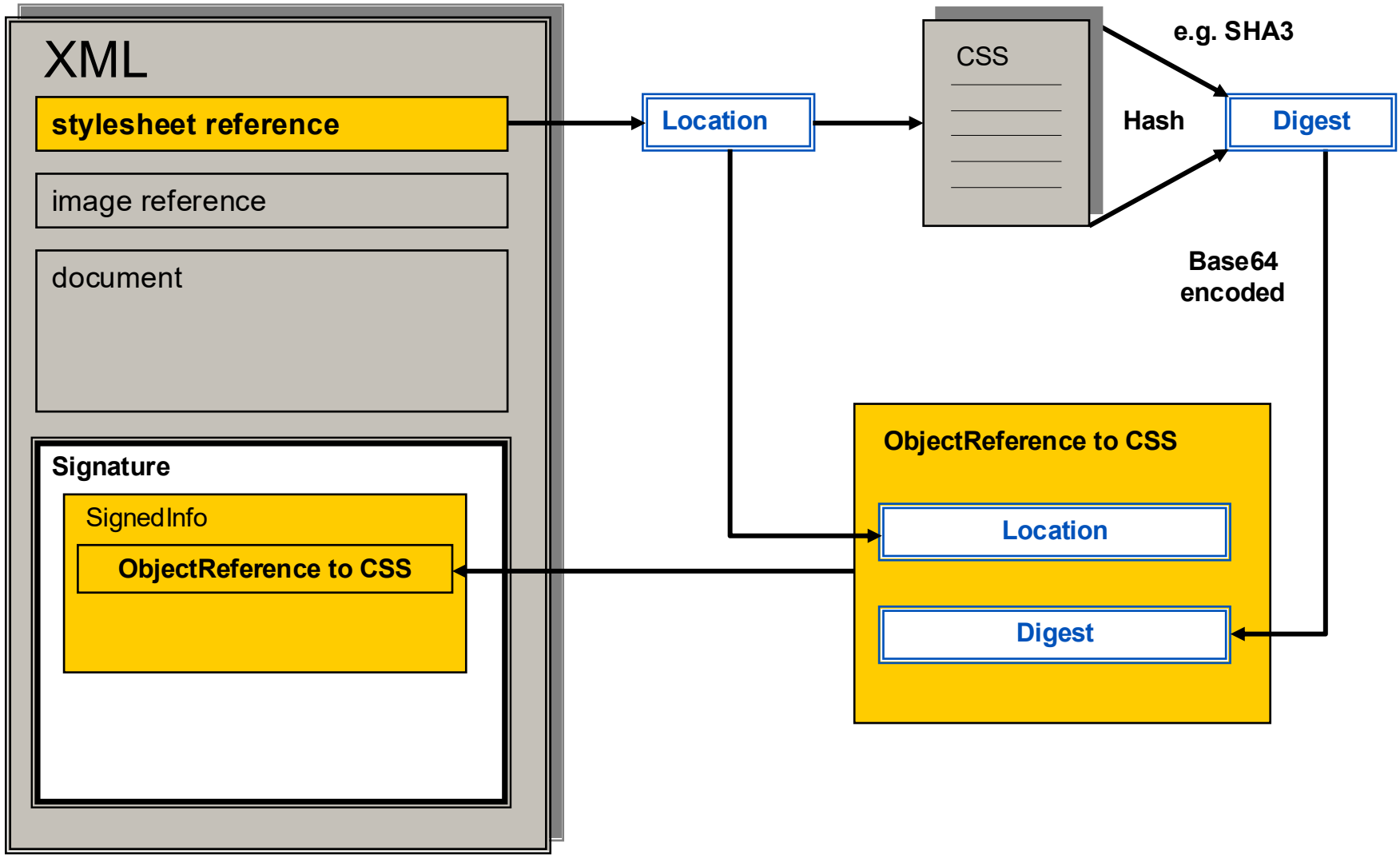




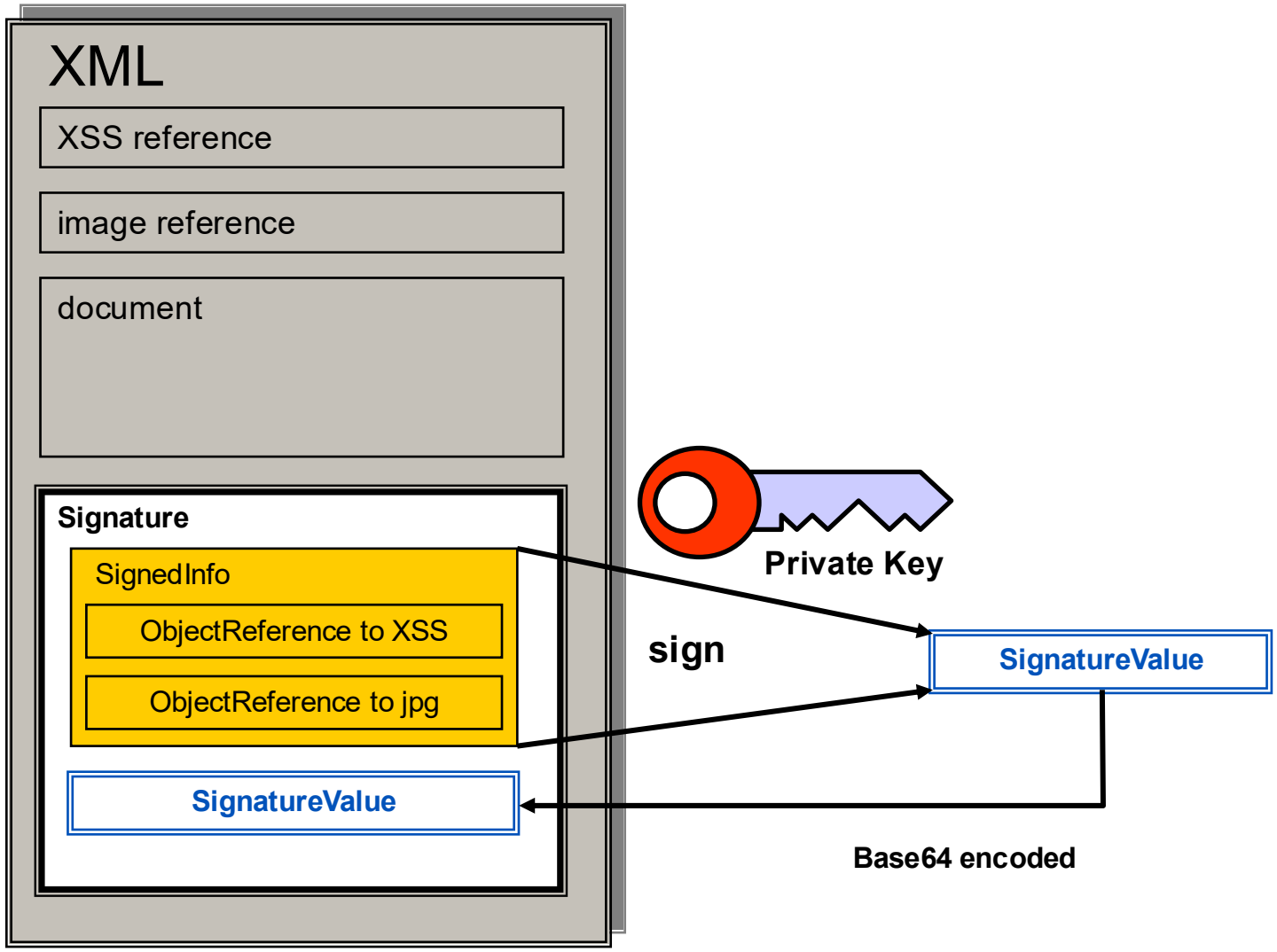
Standard for XML signatures

- Defines rules and syntax for signature
  - of whole XML documents
  - of parts of XML documents
  - any other files
  - literature: http://www.w3.org/Signature/
- Three ways to integrate XML signatures
  - **Detached Signature**: The signature is detached from the document and not embedded in the document.
  - **Enveloped Signature**: The signature is embedded in the document.
  - **Enveloping Signature**: The signature has the function of an envelope. It encloses the entire XML document.

XML-Signature: Digest calculation, reference creation



XML-Signature: Signature of the objekt references



Components of an XML signature

```
<?xml version="1.0" encoding="UTF-8"?>
<Envelope xmlns="urn:envelope">
  <Body> Hello World </Body>
  <Signature xmlns="http://www.w3.org/2000/09/xmldsig#">
    <SignedInfo>
      <CanonicalizationMethod Algorithm="http://www.w3.org/TR/2001/REC-xml-c14n- 20010315#WithComments"/>
      <SignatureMethod Algorithm="http://www.w3.org/2000/09/ xmldsig#rsa-sha1"/>
      <Reference URI="">
        <Transforms>
          <Transform Algorithm="http://www.w3.org/2000/09/ xmldsig#enveloped-signature"/>
        </Transforms>
        <DigestMethod Algorithm="http://www.w3.org/2000/09/ xmldsig#sha1"/>
        <DigestValue>uooqbWYa5VCqcJCbuymBKqm17vY=</DigestValue>
      </Reference>
    </SignedInfo>
    <SignatureValue> KedJuTob5gtvYx9qM3k3gm7kbLBwVbEQRl26S2tmXjqNND7MRGtoew== </SignatureValue>
    <KeyInfo>
      <KeyValue>
        <RSAKeyValue>
          <Modulus>
              4IlzOY3Y9fXoh3Y5f06wBbtTg94Pt6vcfcd1KQ0FLm0S36aGJtTSb6pYKfyX7PqCUQ8wgL6xUJ5GRPEsu9gyz8
              ZobwfZsGCsvu40CWoT9fcFBZPfXro1Vtlh/xl/yYHm+Gzqh0Bw76xtLHSfLfpVOrmZdwKmSFKMTvNXOFd0V18=
          </Modulus>
          <Exponent>AQAB</Exponent>
        </RSAKeyValue>
      </KeyValue>
    </KeyInfo>
  </Signature>
</Envelope>
```

Document to sign

Processing information

Referenced data with processing information and digest

Base64 encoded signature value over the SignedInfo element

Information about the key to verify the signature

XML tags for XML signature

Canonicalization

- XML documents with semantically the same content can be represented in different ways:
  <myelement attr="123"/>
  <myelement attr="123"></myelement>
- This gives the signature a completely different value
- Therefore, a standardized representation is necessary: Canonicalization
- Literature: https://www.w3.org/TR/xml-c14n/

Technische Hochschule Rosenheim

Chapter 3.4: Practical Aspects of Digital Signatures

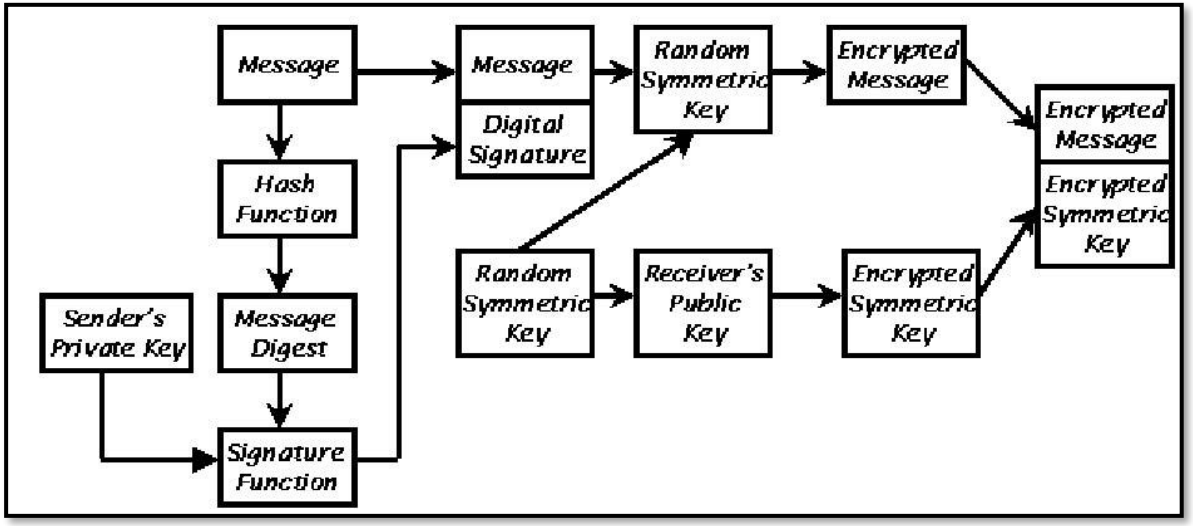
- PKCS#7
- Multiple signatures
- Signature renewal
- Merkle Signatures
- Blockchain

Practical aspects of digital signatures

- Representation problem: With signatures you must see everything you sign
  - **WYSIWYS** (**W**hat **y**ou **s**ee **i**s **w**hat **y**ou **s**ign)
  - There are document formats with content you can't see,
    e.g. macros in Word, JavaScript in web pages.
  - What do I do with such documents?
    - show hidden content
    - eliminate hidden content
    - reformat the document to a format without hidden content
- Signature in combination with encryption
  - first sign then encrypt
  - otherwise, you sign a document that you can not read

PKCS#7 Signature Standard

- Describes structure of encrypted and signed messages
- Multiple formats: Data, Signed-Data, Enveloped-Data, Signed-and-enveloped-Data.
- Process to create a digital envelope around digitally signed data (Signed-and-enveloped-Data) :

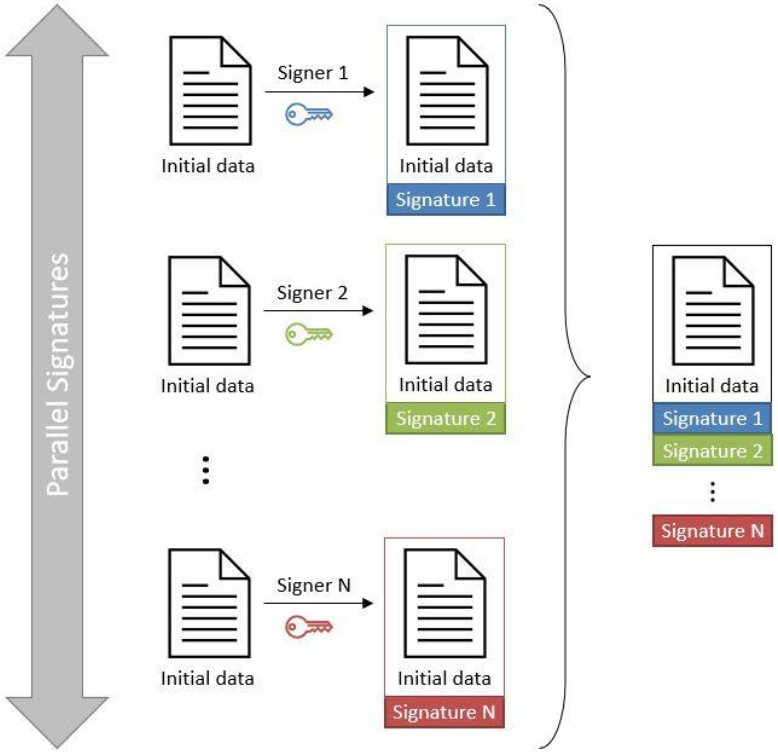


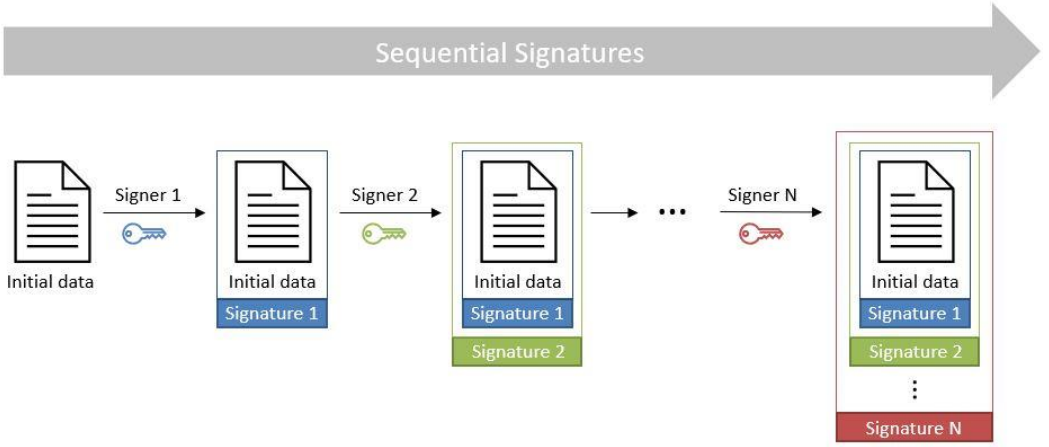
- Further worldwide accepted PKCS published by RSA Laboratories
  - **PKCS#5** (Password-Based Cryptography Standard)
  - **PKCS#10** (Certification Request Syntax Standard)
  - https://en.wikipedia.org/wiki/PKCS

Example for a PKCS#7 Signature

```
SignedData  {
   version           0,
   digestAlgorithms  {
      {1 3 36 3 2 1}, -- OID von RIPEMD-160
      {1 3 14 3 1 18}  -- OID von SHA-1
      },
   encapContentInfo  {
      eContentType  {iso(1) member-body(2) us(840) rsadsi(113549)
                     pkcs(1) pkcs7(7) 1 } -- OID für Data Content
      eContent    [0] "Hello World!"
      },
   signerInfos  {
      {version  1,
       sid      issuerAndSerialNumber {
                   issuer         Alice,
                   serialNumber   3333    -- Zertifikats-Seriennummer
                   },
       digestAlgorithm    {1 3 36 3 2 1}, -- OID von RIPEMD-160
       signatureAlgorithm {1 3 36 3 3 1 2}, -- OID von RSAwithRIPEMD
       signature          'xx..xx' -- RSA-Signatur, 1024 Bit
      },
      {version  2,
       sid      issuerAndSerialNumber {
                   issuer         Alice,
                   serialNumber   4444
                   },
       digestAlgorithm    {1 3 14 3 1 18}  -- OID von SHA-1
       signatureAlgorithm {1 2 840 10045 1}, -- OID von ECDSAwithSHA1
       signature          'yy..yy' -- ECDSA-Signatur, 160 Bit
      }
    }
```

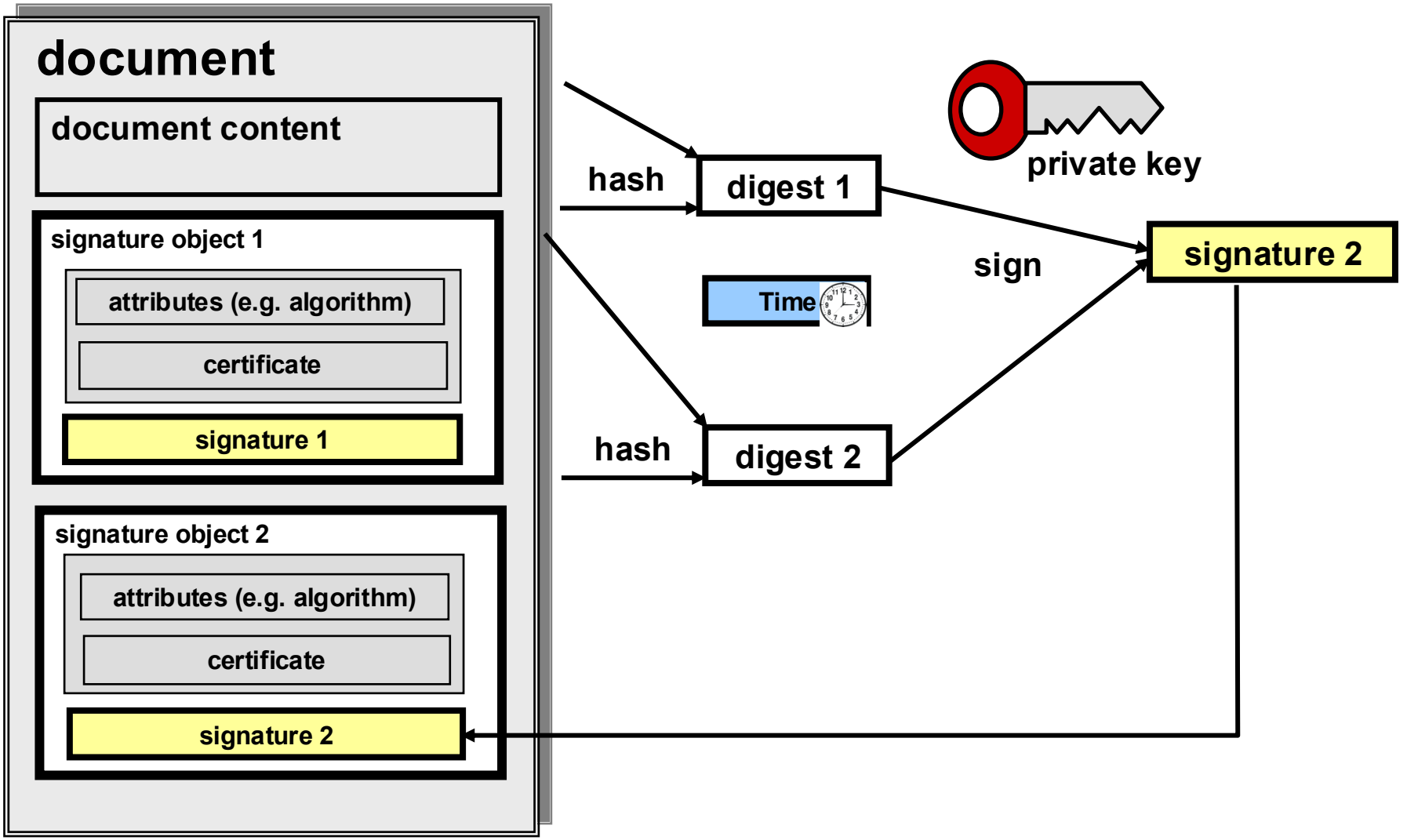
How do multiple signatures work?



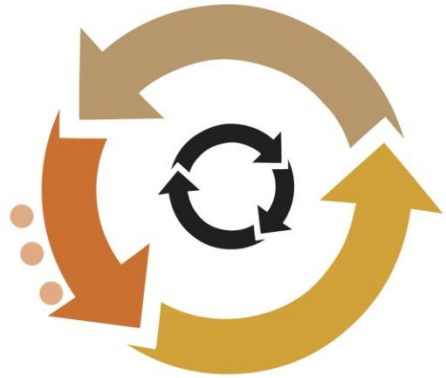


Source: https://dss.nowina.lu/doc/dss-documentation.html#ParallelSignatures

How does signature renewal work?



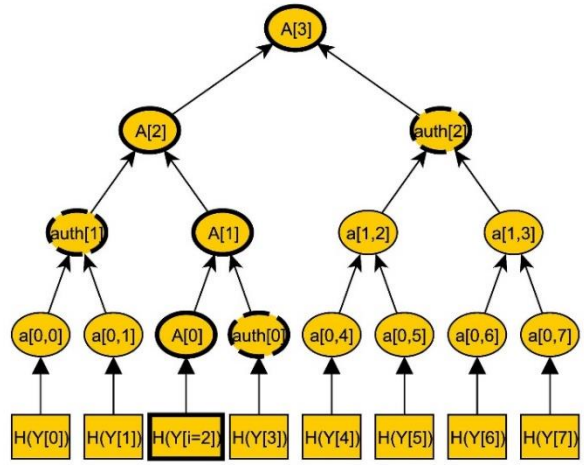
Signature renewal process



- Cause
  - if certificates are no longer listed in the TrustCenter
  - the procedures in the certificate are insecure (hash, encryption)
  - file formats and signature formats are changing
  - laws require verifiability of the signature for a longer period of time
- Process of re-signing
  - verification of the old signature
  - creation of the new signature
    - new signature must include document data and old signature
    - format change of document and/or signature may be necessary
  - process must take place in secure environment
  - process should be certified to ensure legal certainty

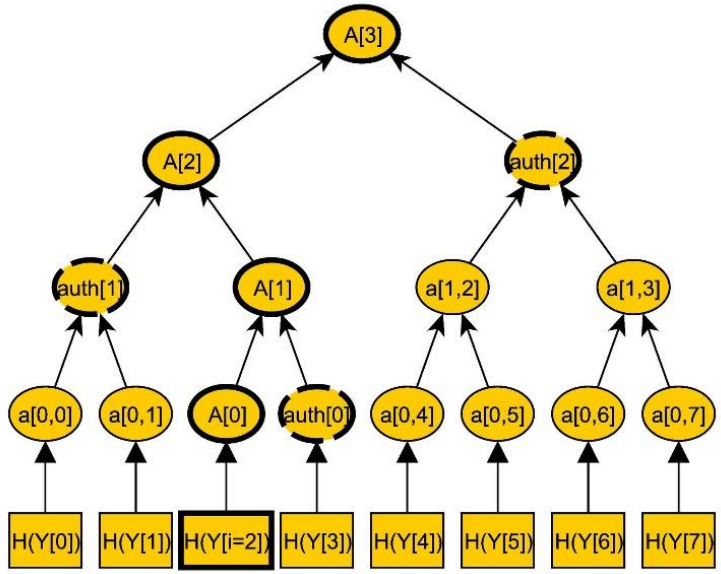
Quelle https://en-academic.com/dic.nsf/enwiki/11462918

Merkle signature scheme



- Merkle signatures are a signature scheme based on hash trees (Merkle tree).
- The public key is the root of the Merkle tree
- The number of signatures per public key is limited (by the number of leaves, this is a power of 2)
- For the signature, the hash values along the path from a leave to the root are also appended.
- If all leaves are used, a new tree must be taken.
- Merkle signatures are resistant to quantum computing
- Merkle trees are used in blockchains for authentication (e.g. Bitcoin)
- Hash trees for securing integrity are more efficient than hash lists

Signature and verification with a Merkle signature scheme



Quelle https://en-academic.com/dic.nsf/enwiki/11462918

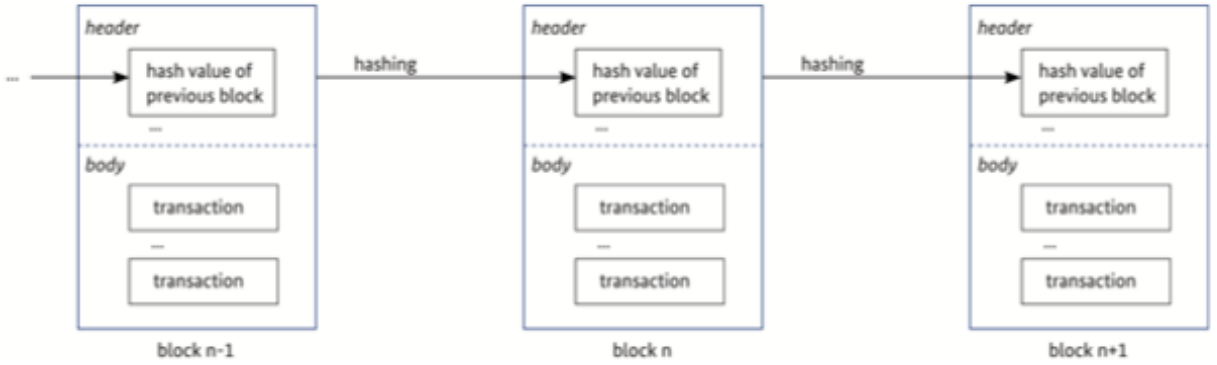
Signature with a Merkle scheme:

1. Generate n key pairs (Xi, Yi) ,
   Xi is private key, Yi is public key
   in the example is i=8
2. Calculate Merkle tree
3. A[n] is public key of Merkle tree
4. Sign message M with Xi, -> sig‘
5. Calculate path from Yi to the root
   Example for i=2
   A[0] = H(Y2)
   A[1] = H(A[0] || auth[0]) = H(A[0] || H(Y3))
   A[2] = H(A[1] || auth[1]) = H(A[1] || H(a[0,0] || H(a[0,1]))
   = H(A[1] || H(H(Y0) || H(Y1))
   A[3] = H(A[2] || auth[2])
6. Signature sig = (sig‘, auth[0], auth[1],auth[2]))

Verification

1. Verify the signature sig‘ with Y2
2. Calculate A[3] from Y2, auth[0], auth[1],auth[2]
3. Verify if public key of Merkle tree is identical with A[3]

The blockchain as an example of application of cryptography



Source: Towards Secure Blockchain – Concepts, Requirements, Assessments, BS Bundesamt für Sicherheit in der Informationstechnik, März 2019)
https://www.bsi.bund.de/SharedDocs/Downloads/DE/BSI/Krypto/Blockchain_Analyse.html

- The chaining of the blocks is protected against manipulation by **hash values**.
- The algorithm for **consensus** (adding a new block to the chain) is usually based on cryptographic methods.
  - e.g. in Bitcoin the block is only accepted if the **miner** finds a hash value for the block that starts with a given number of zeros. For this purpose, the miner may append any number (nonce) until he has a suitable hash.
- **Signatures** with a public key that is not assigned to an explicit user (**pseudonymization**)

How are the goals of IT security implemented in the blockchain?

- **Integrity** is based on hash values
- **Availability** is based on decentralization
- **Confidentiality** is difficult to implement and often not desired.
  - store confidential data on external storage
  - use complex procedures (homomorphic encryption, Trusted Execution Environments TEE, secure Multi-Party Computation sMPC)
- **Authenticity** is based on private signature keys
  - for private blockchains, identification of accounts is desired
  - for public blockchains, pseudonymity is desired

Security goals and design goals are sometimes in conflict

Source: Towards Secure Blockchain – Concepts, Requirements, Assessments, BS Bundesamt für Sicherheit in der Informationstechnik, März 2019)
https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Crypto/Secure_Blockchain.html

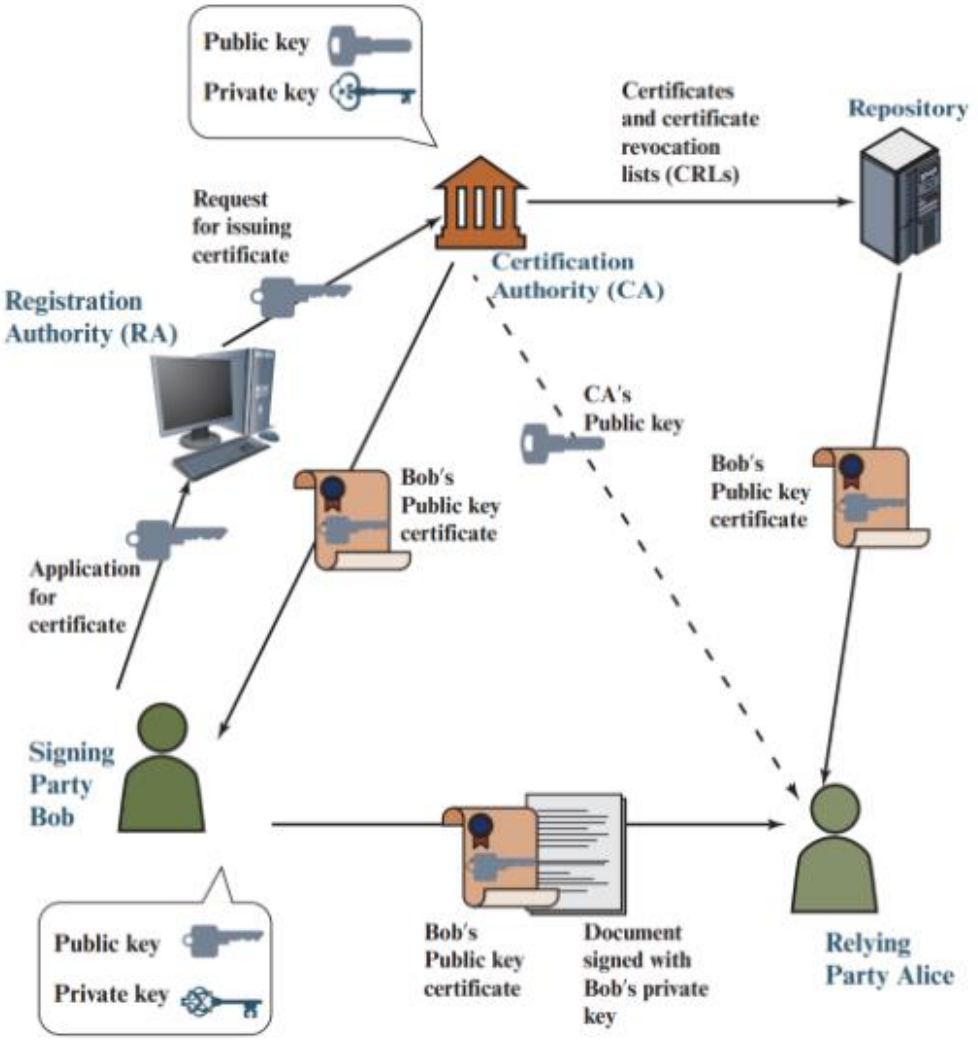
Technische Hochschule Rosenheim

Chapter 3.5: PKI

- Public Key Infrastructure
- Certificates
- European eSignature Directive eIDAS

Public Key Infrastructure (PKI)

- What is a PKI?
  - It provides a confidential and efficient key and certificate management
  - It is a interface for Trust Services (generation, verification, revocation)



Source: Stallings, William. *Cryptography and Network Securit Principles and Practice, Global Edition*. Available from: VitalSource Bookshelf, (8th Edition). Pearson International Content, 2022.

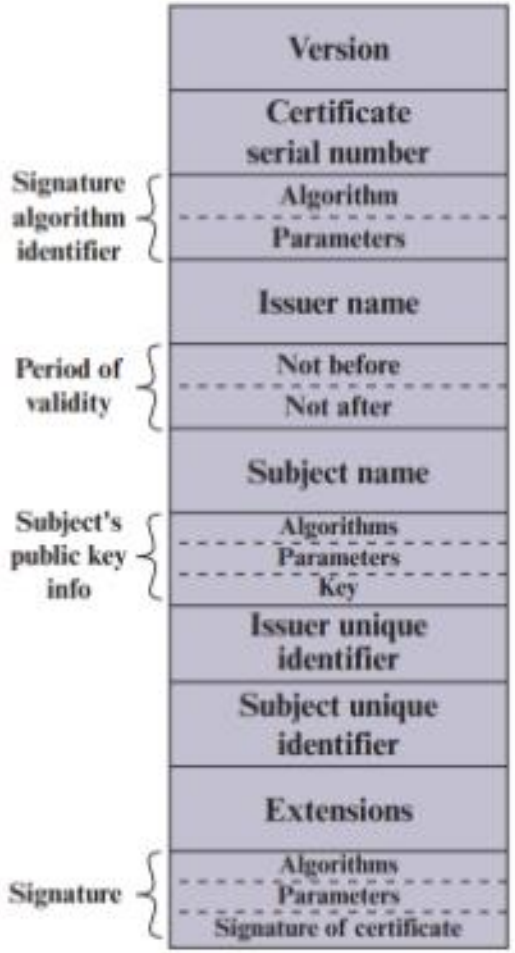
Components of a PKI

- CA Certification Authority
  - creates certificates
- RA Registration Authority
  - interface between CA and subjects (registration office)
  - handles subject identification
- Directory service / Repository
  - contains list of all issued certificates
  - provides revocation-list
- Client unit
  - contains application (PC, Smart phone, ...)
- Personal Security Environment (PSE)
  - Environment in which key is stored (chip card, hard disk, ...)
- Other optional components
  - Time stamp service TSS
  - Revocation instance (REV)
  - Recovery instance

Certificates are digital IDs

- **Problem**: Authentic provision of public keys (man in the middle).
- **Solution**: Certification Authorities (Trust Centers) control the identity of the owner and guarantee the authenticity of the keys
- Certificates
  - have limited lifetime
  - can be revoked
  - are used by protocols such as SSL, S/MIME, IPSec

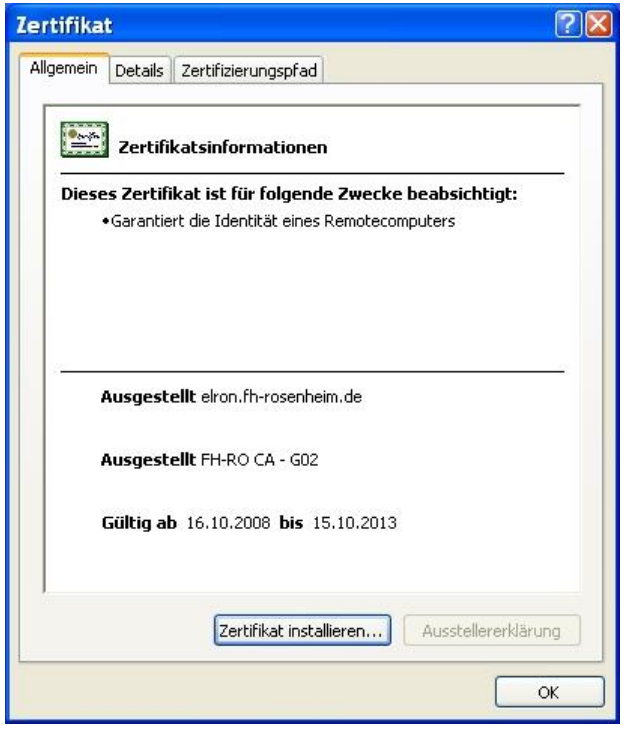
Elements of a X.509 certificate

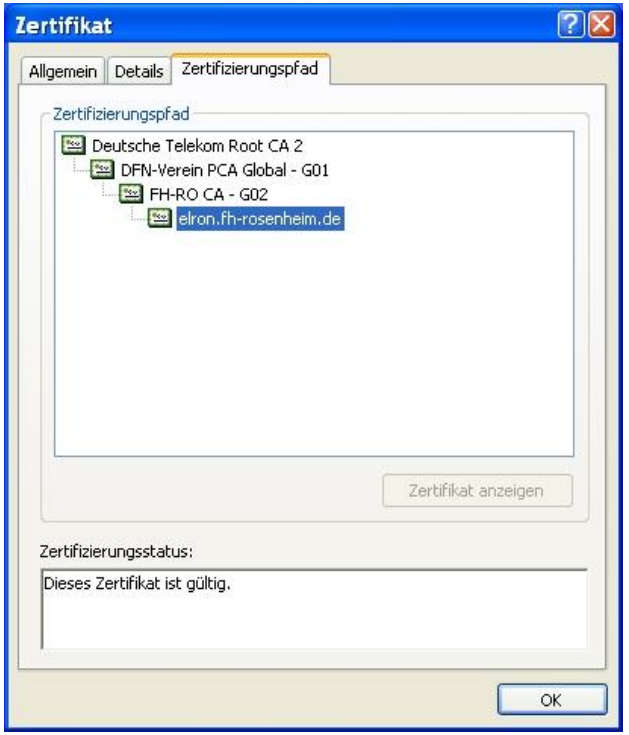


Source: Stallings, William. *Cryptography and Network Security: Principles and Practice, Global Edition*. Available from: VitalSource Bookshelf, (8th Edition). Pearson International Content, 2022.

Standard for certificate format X509v3

- Description language of certificates is ASN.1 (Abstract Syntax Notation)
- Subject Names: stored in format X.500 Distinguished Name
  - CN = Common Name, O = Organization, OU = Organization Untit, C = Country, S = State
- A lot of other attributes can be stored in a certificate (public key, serial number, issuer, …)





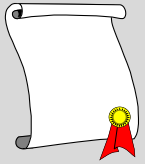
Key and certificate generation

user generates key pair

Keystore

private key

public key



CSR

data (attributes) for the certificate

+

Public Key

Certificate Signing Request

user identifies himself at RA

user generates a CSR and sends it to CA

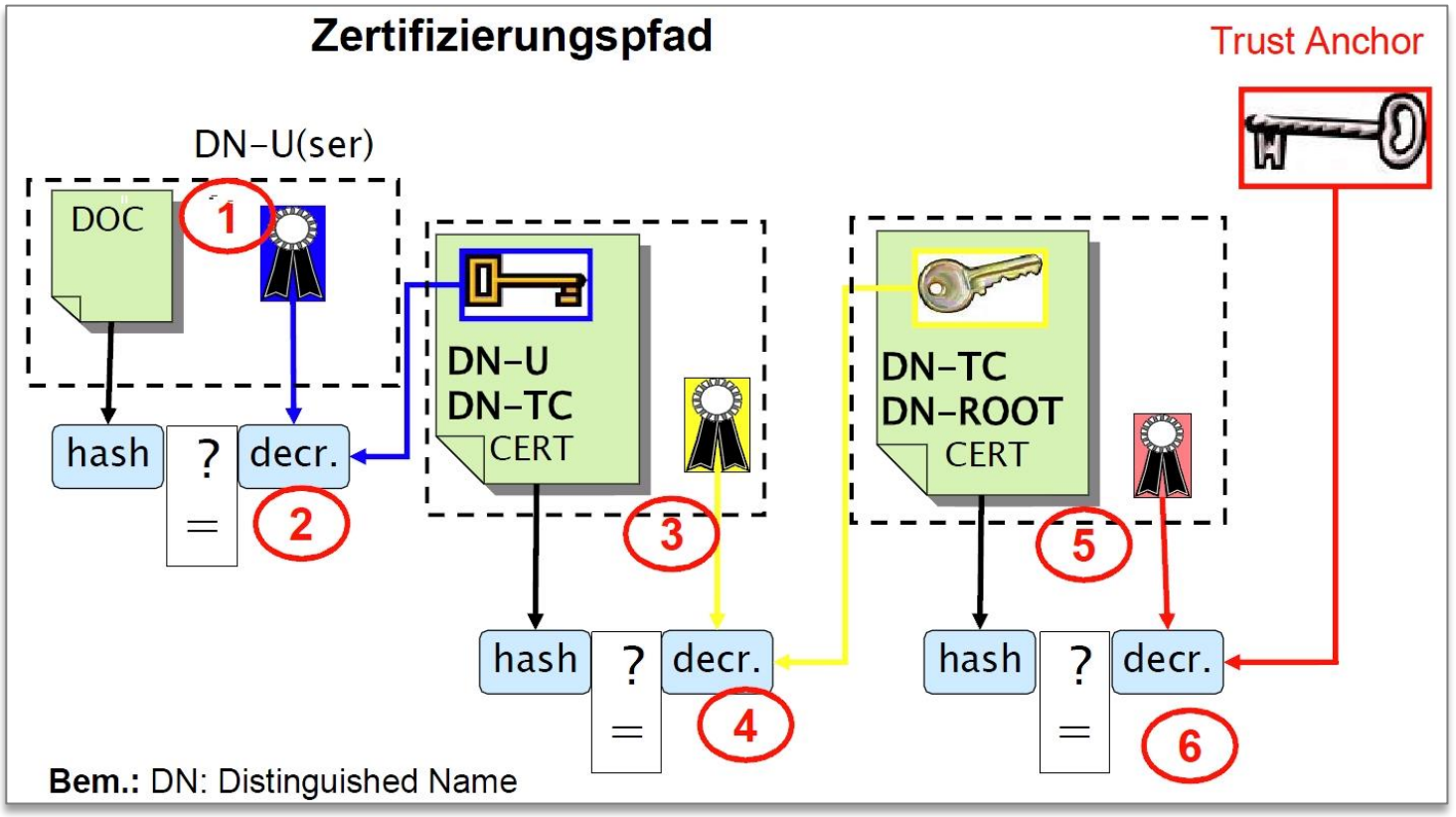
CA sends certificate to applicant

RA

RA confirms identity of applicant and public key

CA

Verification of a signature and the certificates.

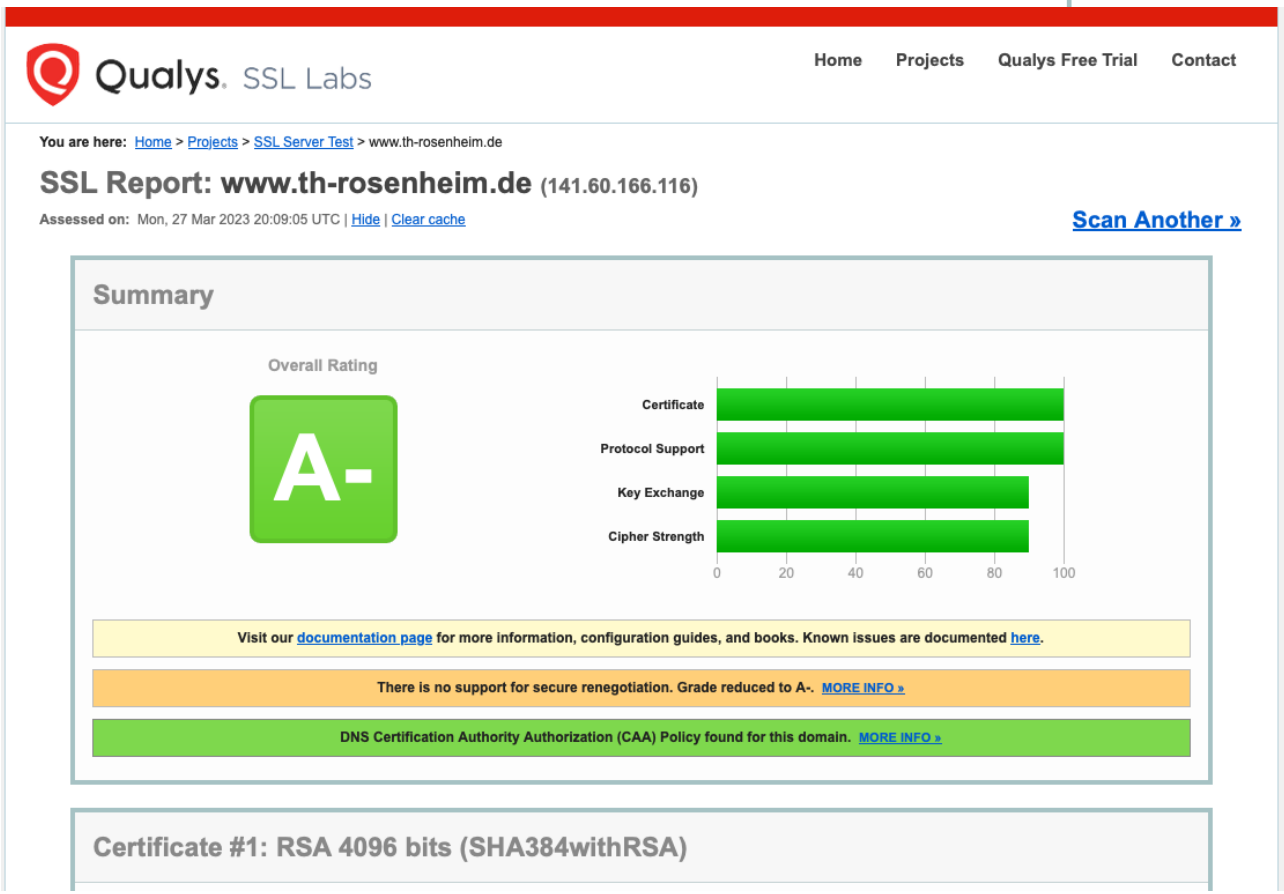


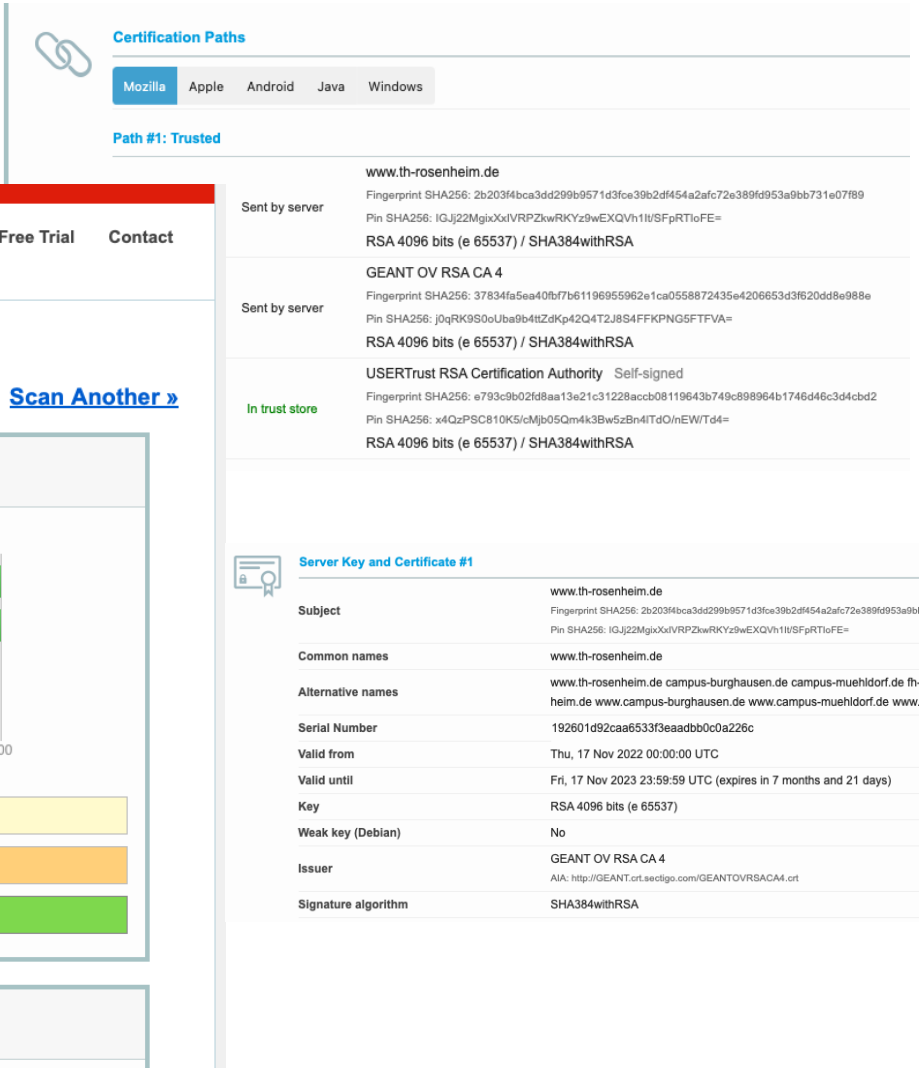
Source: Claudia Eckert, TUM

- Get all certificates of the chain (often via LDAP Lightweight Directory Access Protocoll)
- Check validity period and signatures of the certificates
- Check revocation list (**OCSP** Online Certificate Status Protocol)

Online tool for verifying certificates.

https://www.ssllabs.com/ssltest/

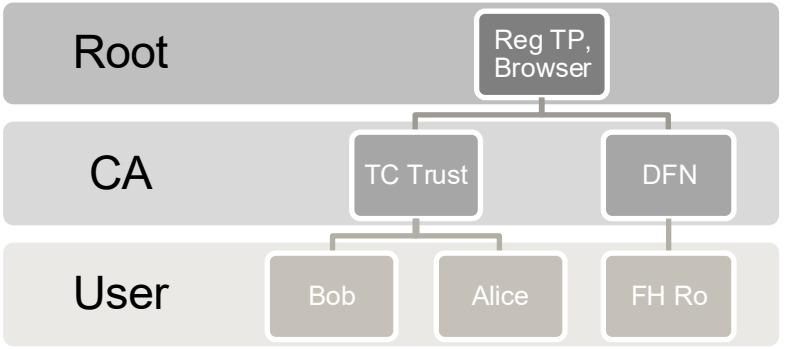




Certification models

- Web of Trust
  - + simple and flexible in use
  - + many potential certificate chains
  - - no evidential value, or only with difficulty to obtain
  - - finding a trustworthy path is more complex

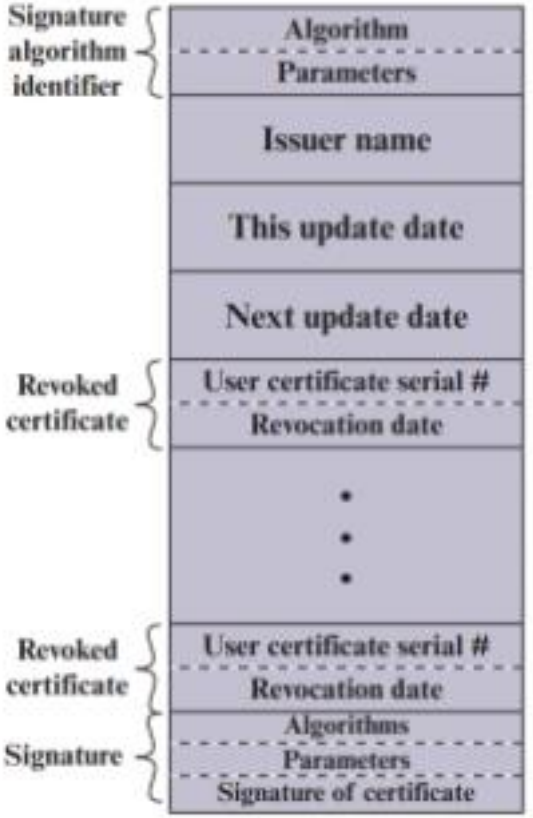
- Hierarchical certification
  - + clear structures and accountability
  - + evidential value in case of dispute
  - - overhead due to organizational structure



- Example for a free certification authority https://letsencrypt.org

CRL Certificate Revocation List

- Motivation: In case of loss or theft (copy), a key must be blocked to warn other users
- Properties of the CRL
  - list of serial numbers of all revoked certificates
  - IETF standard
  - stored at directory service of CA
  - signed and timestamped by CA
  - frequently updated
- Issues
  - actuality
  - size (can get very large)
  - Distribution to the applications, how can clients access the list?
    - Download CRL of CA
    - Perform Online Status Check (OCSP Online Certificate Status Protocol)



European eSignature Directive eIDAS

- **E**lectronic **ID**entification **A**uthentication and trust **S**ervices (since 1.7.2016)
- EU regulation on electronic identification, electronic signatures and trust services for electronic transactions in the internal market
  http://eur-lex.europa.eu/eli/reg/2014/910/oj
- Three level of signatures
  - **Simple** electronic signatures
    - Data in electronic form attached to the document
  - **Advanced** electronic signatures (AdES)
    - uniquely linked to and capable of identifying the signatory
    - linked to the document in a way that any subsequent change of the data is detectable.
  - **Qualified** electronic signatures
    - created by a qualified signature creation device (QES)
    - and is based on a qualified certificate for electronic signatures
    - it is equivalent to a handwritten signature.

Components of the eIDAS regulation

- **Electronic identification** (identity card with eID functions)
- **Trust services** (providers of qualified services)
  - support creation and verification of **electronic signatures** (natural persons, declaration of intent), **electronic seals** (legal persons, proof of origin), electronic **time stamps**
  - deliver electronic registered mail
  - issue certificates for web site authentication
  - have to go through certification process themselves
- This enables creation of electronic documents with
  - electronic signature as legitimate proof
  - authentication of the document by electronic seal
  - proof of creation by time stamp
  - confirmation of receipt by electronic delivery services

Applications of digital signatures

- Digital identity card
- Archiving of documents (with time stamp)
- Paperless invoices, reminders for invoices
- Public authorities (e-government, land registry)
- Electronic tax declaration (Elster)
- Pension account information
- Communication with patent court, patent office
- Digital banking transactions
- Electronic signatures with cell phone or tablet

Summary checksums and digital signatures

- Cryptographic checksums such as MAC enable the authentication of data
- Digital signatures are a combination of hash value calculation and asymmetric encryption
- During implementation, many aspects must be considered (multiple signatures, signature renewal, canonicalization)
- In practice, digital signatures often require a great effort for hardware, software and process redesign
- A PKI is the basis for certificates and public key management
- A PKI enables digital signatures and confidential communication
- A certificate is a digital identity card that should be issued by a trustworthy trust center.
- Verification of a certificate requires checking the certificate chain, the CRL and the content of the certificate.